ETL Developer Guide

How to structure your ETL package so the platform can run it correctly

1. What is an ETL Package?

An ETL package is a **.zip** file uploaded to the platform. It contains Python scripts, a config file, and a requirements file. The platform extracts it, installs dependencies in an isolated virtual environment, patches the config with any user overrides, and runs the entry point script.

The platform never modifies the original ZIP. Every execution gets a clean isolated copy of the code in its own working directory.

2. Required Files

File Type	Required	Description
.py	YES	Entry point: the script the platform runs
.json / .yaml	YES	All runtime parameters (paths, dates, settings)
.txt	Recommended	Python dependencies (requirements.txt)
Other	Optional	Helper files, must be inside the ZIP

3. ZIP Structure

- Do not include `.venv/`, `venv/`, `__pycache__/`, `.git/`. They are stripped automatically.
- Keep the ZIP under a reasonable size. Large input data files should not be inside the ZIP. Reference them by path in the config.
- Every helper module, SQL file, template, or mapping must be inside the ZIP.

Example:

```
my_etl.zip
    ■■■ main.py
    ■■■ config.json
    ■■■ requirements.txt
```

4. Configuration File Rules

The config file is the contract between the script and the platform. It is read, patched with user overrides at launch time, and written back to every copy inside the working directory before the script runs.

General rules:

- Any filename is valid. `.json` and `.yaml` are fully supported.
- All paths must come from the config. Never hardcode paths inside the script.
- Every folder the script writes to must appear in the config as an output path.
- Keys with spaces in their names can cause parsing issues. Avoid them.
- Date fields use ISO format: `YYYY-MM-DD` or `YYYY-MM`.
- Booleans must be real JSON booleans: `true` / `false`, not strings.

Folder Block:

Use a Folder Block when an input folder must contain specific files that the platform should verify before launch.

```
"input_folder": {
    "path": "C:/Data/Source/",
    "files_number": 10,
    "check_mode": "both",
    "file1": "report.xlsx",
    "file2": "parc*S*.xlsx",
    "file3": "*.csv",
    "file4": "-"
}
```

check_mode options:

Value	Behaviour
count	Checks total file count matches files_number
files	Checks each fileN entry exists
both	Checks count AND each fileN entry
auto	Uses files if fileN entries exist, count otherwise (default)

fileN pattern syntax:

Pattern	Matches
report.xlsx	Exact filename, case-insensitive
report*	Any file starting with report
*_final.xlsx	Any file ending with _final.xlsx
*parc*S*	Any file containing both parc and S
*.csv	Any file with that extension
.xlsx	Shorthand for *.xlsx
-	Placeholder, not set yet. Warning shown but launch not blocked

5. Script Rules

- Read config with `open("config.json")`.
- Wrap logic in `main()`, call under `if __name__ == "__main__"`.
- Exit code 0 = success, anything else = FAILED.
- **Write at least one output file.** No output file = FAILED even on exit 0.
- Print progress to stdout. It appears in real time in the platform UI.

6. Path Classifications

During the launch flow, every path-like config key gets classified as:

Classification	Platform behaviour

input	Checks it exists before running and syncs it into the working directory
output	Creates the directory automatically and scans it for result files after the run
skip	Ignored. No check, no sync, no scan

7. Output Files

The following extensions are collected and registered after each run:

```
.xlsx .xls .csv .pdf .zip .json .txt .parquet
```

An execution that produces no registered output file is marked FAILED, even if the script exits with code 0. A pipeline that completes without producing data is treated as a failure.

8. Outcome Detection

The platform determines the final status using three criteria applied in order:

Priority	Criterion	Result
1	Non-zero exit code	FAILED
2	Python traceback in stderr	FAILED
3	No output files produced	FAILED
	All criteria pass	SUCCESS

9. requirements.txt

- Always include one, even if nearly empty.
- Pin versions for reproducibility: `pandas==2.1.4`
- The platform creates one shared virtual environment per ETL and reuses it across all executions.
- To apply changes to requirements.txt, the administrator must trigger a venv rebuild from the ETL management interface. The next execution will then reinstall dependencies.

10. Quick Checklist

	Item
[ ]	ZIP contains entry point .py file
[ ]	ZIP contains config file (.json or .yaml)
[ ]	ZIP contains requirements.txt
[ ]	All paths come from config, none hardcoded
[ ]	Every output folder is a config key
[ ]	Script writes at least one output file
[ ]	Script exits with code 0 on success
[ ]	No .venv / __pycache__ / .git inside ZIP

[ ]	Large input files are outside the ZIP, referenced by path
[ ]	Dependency versions pinned in requirements.txt